ChunkTevict: Chunk-Granularity TevictRanges for Geo-Distributed Object Storage

Anonymous Authors
Anonymous Institution

***Abstract*—Public-cloud object stores bill bytes stored over time and bytes moved across regions, so a geo-distributed application must repeatedly choose between keeping a remote replica and fetching it again. SkyStore answers this question with TevictRanges, an adaptive time-to-live (TTL) that learns when a region will next read an object. TevictRanges, however, treats every object as an opaque byte range: a GET miss copies the whole object, and a cached replica is billed at full size until it expires, even when most of its content already lives in that region. We present *ChunkTevict*, which keeps SkyStore's interval histogram and T_{evict} estimator unchanged but moves placement, eviction, transfer, and GB·h accounting from objects to unique content-defined chunks in each region. We bind the keys of five IBM Object Store traces to real versioned content, chunk it once offline, and replay SkyStore's two-, three-, six-, and nine-region settings, recording storage, transfer, and simulated GET RTT in one run. In two regions, object-level SkyStore costs 2.18–$11.80\times$ more than ChunkTevict, and storage accounts for 88–97% of the gap. Across SkyStore's four three-region workload types, SkyStore averages $6.40\times$ ChunkTevict with a fixed base region and $5.00\times$ under free placement, so removing the immortal origin replica does not close the gap. The savings depend on how much content repeats across objects; we discuss this and other threats to validity.**

***Index Terms*—Geo-distributed storage, object placement, TTL, content-defined chunking, cost optimization**

I. Introduction

With the prevalence of public-cloud object storage, geo-distributed applications pay for three things: bytes stored over time, bytes transferred across regions or providers, and API calls. When clients in several regions read and write the same bucket, a local replica avoids a future cross-region miss but accrues storage cost until it is evicted. Storing every object everywhere wastes GB·h; refetching on every read wastes egress. SkyStore [1] casts this trade-off as a caching problem without a capacity limit: replicate on read, and evict a replica when keeping it is predicted to cost more than fetching it again. Its eviction policy, TevictRanges, learns a per-region TTL from a histogram of past inter-access times and beats Always Evict, Always Store, and a fixed break-even TTL (T_{even}) on IBM Cloud Object Store traces [2].

TevictRanges, like the object-level Optimal that SkyStore compares against, makes every decision per *object*. A GET miss materializes the full object in the issuing region; the histogram is weighted by the object's logical size; every live replica is billed at full size. That matches the S3 API, but not the bytes that actually differ between objects. Content-defined chunking (CDC) [3], [4] splits data into variable-length chunks identified by fingerprint, and versioned datasets—source trees, packages, images—share most of their chunks. When two objects share a chunk, an object-level clock stores that chunk twice under two independent leases, and a miss on the second object transfers it again even though it is already in the region. In other words, the estimator is fine; the unit it is applied to is too coarse.

Therefore, we propose **ChunkTevict**. ChunkTevict keeps TevictRanges' interval histogram and T_{evict} estimator, and SkyStore's TTL selection rule, but applies them to each unique chunk in each region. The histogram learns from an object's *unique* chunk bytes. A miss fetches only the chunks that are not local, each from its cheapest holder, and each chunk carries its own lease. SkyStore's fixed-base (FB) and free-placement (FP) semantics are preserved: an FB origin copy is pinned forever, and under FP the last remaining copy of a chunk is never dropped.

The main contributions of this paper are summarized as follows.

- First, we analyze the limitation of object-granularity TevictRanges. When object content overlaps, SkyStore and even its object-level Optimal store and move the same chunks repeatedly; on two regions, Optimal remains 2.27–$11.73\times$ more expensive than ChunkTevict.
- Second, based on this observation, we propose ChunkTevict, which changes the billing and lease unit from objects to unique chunks while leaving SkyStore's estimator and TTL rule untouched.
- Third, we bind five IBM traces to real versioned content, chunk it offline, and replay SkyStore's evaluation (its Fig. 5, Tables 3–5, and Fig. 6) with the same baselines. SkyStore costs 2.18–$11.80\times$ ChunkTevict in two regions, $6.40\times$ on average across four three-region workload types, and $5.00\times$ under three-region free placement.

The rest of the paper is organized as follows. Section II reviews geo-placement, TTL caches, and chunking. Section III analyzes object-granularity TevictRanges. Section IV presents ChunkTevict. Section V reports experimental results, and Section VI concludes.

II. Related Work

Geo-distributed object stores minimize a mix of latency, availability, and price. We group related work by whether the system bills *objects* or *content chunks*.

A. Geo-Distributed Object Placement

SPANStore [5] and Volley [6] place data across cloud regions to trade latency, availability, and price. Uluyol *et al.* [7] study near-optimal latency–cost trade-offs for geo-storage. SkyStore [1] specializes the problem to object GET/PUT traces with public-cloud list prices. Its two-region Fig. 5 compares Always Evict, Always Store, T_{even}, TTL-CC, EWMA, and AWS Multi-Region against TevictRanges; Table 3 reports cost versus an offline object-level Optimal; Table 4 (three-region) uses JuiceFS instead of AWS Multi-Region. Our evaluation uses that same baseline set and changes only the billing unit to unique chunks.

SkyStore's traces expose timestamp, operation, region, key, and size—not content—so any policy driven by those fields treats an object as one billing unit. ChunkTevict is an adaptation of TevictRanges to a precomputed chunk index, not a new multi-cloud overlay.

B. TTL Caches

TTL-based caches [8] set a lifetime from an estimate of the next request, then let the object expire without a capacity LRU. TevictRanges is a geo-storage TTL: the lifetime of a replica at destination d is chosen so that storing until the predicted next access at d is cheaper than paying some network edge on the next miss. ChunkTevict reuses that estimator unchanged. The only change is the entity that carries the TTL: a unique chunk in a region instead of an object.

C. Chunking and Deduplication

Low-bandwidth file systems [3] and Rabin fingerprints [4] made content-defined chunking standard, and backup and primary-storage deduplication store each unique chunk once. Resemblance detection and delta compression [9]–[11] additionally remove redundancy between similar but non-identical chunks, at the cost of feature metadata. These systems decide *what* to store; they do not decide *where* and *for how long* a copy should live across priced regions. ChunkTevict uses exact chunk identity from an offline CDC index, does not encode deltas, and does not tune chunk size at runtime. Its contribution is to drive a geo-TTL with that index.

III. Problem Statement

In this section, we discuss the limitation of applying TevictRanges at object granularity.

A. TevictRanges in Brief

SkyStore stores an object in the region that writes it and replicates it to a region that reads it, paying the cheapest egress edge. Each replica then gets a TTL. TevictRanges maintains, per destination region, a histogram of time-to-next-access weighted by object size $|o|$; $T_{\text{evict}}(s \to d)$ is the horizon that minimizes predicted storage-plus-refetch cost at d when the miss would be served from s. On a miss, SkyStore takes the minimum T_{evict} over other replicas whose own lease still covers that horizon; if there is no other replica, it falls back to a median break-even time. Always Evict never keeps a remote copy, Always Store never evicts, and T_{even} uses one break-even time from list prices. SkyStore's Optimal knows future object accesses but still decides per object.

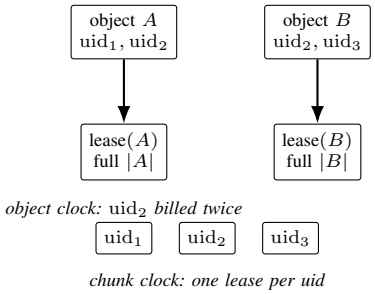


Fig. 1. Object-level TevictRanges keeps independent leases for A and B even when they share uid_2. ChunkTevict stores uid_2 once per region.

B. Observations

CDC maps object o to a sequence of chunk identifiers (uids) with lengths $\ell[\text{uid}]$. In a region, the bytes that occupy storage are $\sum \ell[\text{uid}]$ over uids with a live lease, not $\sum |o|$ over live objects.

a) Observation 1 (objects share chunks).: Once trace keys are bound to versioned content (Section V-A), most chunks written by a PUT already exist in the issuing region. In two-region replay, 82.3% (T15), 97.0% (T29), 87.9% (T78), and 81.3% (T79) of PUT chunk references are already local. T65 is colder on PUT (39.7%) but its GETs hit local chunks 99.8% of the time. An object-level PUT or cached replica is nonetheless billed at full size.

b) Observation 2 (object Optimal is not byte-optimal).: Table III shows that two-region object-level Optimal is $3.60\times$, $11.73\times$, $2.27\times$, $6.51\times$, and $6.23\times$ ChunkTevict on T15/T29/T65/T78/T79. Knowing future object accesses does not help when the bill is per object: Optimal still stores and moves whole keys.

c) Observation 3 (the extremes fail differently).: Always Evict explodes on hot read traces: $27.39\times$ ChunkTevict on two-region T65, 96.1% of it transfer. Always Store explodes on large, long-lived objects: $10.40\times$ on T79, 97.9% of it storage. TevictRanges sits between the extremes at object granularity; ChunkTevict uses the same rule on unique chunks.

These observations suggest that a finer *clock*—not a better estimator—is what object-level geo-TTLs are missing.

Fig. 1 sketches the mismatch. Object A and object B share uid_2. An object miss on B recopies uid_2 and starts a second lease, even if uid_2 is already live for A. ChunkTevict transfers only missing uids and keeps one lease per uid per region.

IV. Design

In this section, we first describe the offline chunk store. Next, we present TevictRanges as a pure estimator and the GET/PUT paths (Algorithm 2). Finally, we briefly describe the implementation.

A. Offline Chunk Store

Simulation never re-runs CDC on the critical path. A one-time index maps each object key to a packed array of uids and a logical size, plus a global length table $\ell[\text{uid}]$. The histogram weight of object o is

$$w(o) = \sum_{u \in \text{unique}(\text{uids}(o))} \ell[u], \tag{1}$$

i.e., duplicate uids inside one object are counted once [3]. If o is missing from the index, the policy falls back to the trace size field. Each chunk reference occupies a fixed 40-byte control-plane fingerprint in the manifest; we do not fold that metadata into the storage bill (Section V-C7).

B. TevictRanges as a Pure Estimator

Placement of a new object (which regions may hold a first copy) is unchanged: ChunkTevict calls TevictRanges `place()` and `update_past_requests()`. The latter is fed a synthetic request whose size is $w(o)$ rather than $|o|$, so the histogram learns unique bytes. TTL selection is SkyStore's `calculate_ttl` [1], applied to the uid's current replicas rather than the object's. Transfer pays the cheapest holder; the lease is SkyStore's T_{evict} over replicas that still cover the horizon (or are immortal). If no other holder exists, the policy uses the same median T_{even} fallback.

The estimator itself is SkyStore's: `TevictRangesV2.get_tevict`(src, dst, t) still scans the interval histogram at dst and returns seconds. ChunkTevict does not edit that routine. What changes is the *clock object*: SkyStore's `calculate_ttl` walks other *object* replicas, takes $\min_i T_{\text{evict}}(i \to R)$ among copies whose remaining lease covers $t + T$ (or are immortal), and hangs one TTL on the object in region R. ChunkTevict runs that same min over regions that currently hold the *uid*, and hangs the lease on that uid in R (`RegionChunkState.chunk_exp`). Histogram updates still go through `update_past_requests`, but the synthetic request size is $w(o)$ rather than $|o|$. FB origin PUTs pin every uid at R forever. FP PUTs set a TTL only on newly installed uids (already-local chunks are not refreshed), matching SkyStore's skip-if-stored rule.

C. GET and PUT Paths

Algorithm 2 summarizes one request at issuing region R and time t. PUTs incur no transfer in this model (the client already has the bytes).

On GET miss, object-level SkyStore pays for the whole object from one replica-holding region and sets one object TTL. ChunkTevict runs that same pay-and-lease pair *per missing uid*, so different chunks of one object may come from different regions and expire independently.

D. Last-Copy Freeze (FP only)

When a uid's lease expires and it is the last copy, ChunkTevict does not delete it: the expiry becomes ∞ and the uid is marked pending. A later GET hit on that copy clears pending and installs a new finite TTL (SkyStore TevictRanges does the same at object granularity). A later PUT or GET miss that creates a second copy elsewhere expires the pending copy immediately. FB never freezes: the origin pin is already immortal.

Algorithm 1 ChunkTevict replay of one request.

Input: object key, op $\in \{\text{GET}, \text{PUT}\}$, region R, time t.
Output: storage/GB·h updates, transfer charges, per-uid leases.

$U \leftarrow \text{unique}(\text{uids}(\text{key}))$
if op $=$ PUT **and** FB **then** pin every $u \in U$ at R (TTL $= \infty$); **return**
if op $=$ PUT **and** FP **then**
  install missing $u \in U$ at R; do *not* refresh already-local uids
  for each newly installed u: TTL $\leftarrow$ SkyStore `calculate_ttl` on holders of u (else median $T_{\text{even}}/3$)
  release last-copy freeze of u in other regions; **return**
// GET
update TevictRanges histogram at R with weight $w(\text{key})$
for each local non-pinned $u \in U$: refresh TTL $\leftarrow$ SkyStore `calculate_ttl` on holders of u
for each missing $u \in U$: pay cheapest holder; install u at R; TTL $\leftarrow$ the same `calculate_ttl`
FP: if a second copy of u now exists, expire any pending last-copy freeze elsewhere

Fig. 2. ChunkTevict applies SkyStore's TevictRanges `calculate_ttl` per unique chunk.

E. Implementation

The replay sits alongside SkyStore's simulator and does not modify the TevictRanges core. The chunk store memory-maps packed uid arrays and a global length table. Each region keeps a dense per-uid lease vector, a pin mask, and an expiry heap. Transfer charges use SkyStore's directed price graph (storage USD/GB·h and pairwise egress). CDC is performed once offline (FastCDC over a content pool bound to keys by a manifest); the replay only looks up uids. Manifest metadata is 40 B per chunk reference and is excluded from the dollar bill.

V. Evaluation

Following SkyStore [1] §6, we answer five questions:

1) How much cheaper is ChunkTevict than SkyStore's two-region baselines, object-level SkyStore, and Optimal?
2) Is the gap storage or transfer, and do chunk hits explain it?
3) Do the savings hold across SkyStore's three-region workload types and as the number of regions grows?
4) Do they survive free placement, where no origin copy is pinned?
5) What does the same replay say about simulated GET RTT and replay time?

We organize the section as in [11]: datasets, metrics, then results discussed figure by figure.

A. A. Datasets

To evaluate the method, we use the five complete IBM Object Store traces of SkyStore Fig. 5 (T15, T29, T65, T78, T79) [1], [2]. The traces record timestamp, operation, region, key, and object size, but not payload. We therefore bind every key to real content once, offline. A manifest maps each key to a file of matching size class from a pool of versioned public source releases (files extracted from many

consecutive Linux kernel versions, plus PostgreSQL and other archives for large objects). The pooled bytes are chunked with FastCDC (minimum 2 KiB, average 8 KiB, maximum 64 KiB) and fingerprinted with BLAKE3; the replay only looks up the resulting chunk ids. The amount of cross-object sharing is therefore a property of this binding, not of the original IBM workload; we return to this point in Section V-D. Two-region runs use SkyStore type E (base and cache). Three-, six-, and nine-region runs use types A–D (uniform, region-aware, aggregation, replication).

T15 is write-heavy with many one-hit objects: Always Evict and TevictRanges stay close at object granularity, while Always Store and AWS Multi-Region pay a second replica that is rarely reread.

T29 is also write-heavy, but objects are tiny and the trace exceeds 12 million requests. GETs miss often at chunk grain (GET hit 43.8% in two regions), yet PUTs reuse 97.0% of unique chunks, so object-level clocks still store and move whole keys.

T65 is read-heavy and hot. Almost every GET chunk hits locally (99.8% in two regions), so Always Evict pays transfer on every miss while Always Store and SkyStore stay cheap—and still several times ChunkTevict, because they keep full objects.

T78 is read-heavy at high volume (2.41 million operations). Always Evict is catastrophic; T_{even} and TevictRanges beat Always Store, as in SkyStore §6.3, but remain far above the unique-chunk bill.

T79 has large objects and a long GET tail. Always Store and AWS Multi-Region keep full replicas for the whole trace; ChunkTevict stores only unique uids.

B. B. Metrics and Setup

We evaluate efficiency with two primary metrics, following [11]: monetary cost without request charges, and overall replay time. Cost is storage plus transfer at public list prices, matching SkyStore's "cost without request." We report absolute USD and the ratio $(\text{policy cost})/(\text{ChunkTevict cost})$. The same replay records simulated GET and PUT RTT from the directed transfer graph (same-region self-loop = 1 ms). A GET miss pays the RTT of the chosen edge; ChunkTevict takes the maximum over missing chunks fetched in parallel. This is a lookup on the price graph, not SkyStore §6.6 end-to-end client latency on a live three-cloud deployment. PUT RTT is 1 ms for every policy (local write).

Policies follow SkyStore §6.2.2–§6.4. Two-region Fig. 5 plots Always Evict, Always Store, T_{even}, TTL-CC, EWMA, and AWS Multi-Region Bucket; Table 3 separately reports cost versus SkyStore's offline object-level Optimal. Three-region Table 4 replaces AWS Multi-Region with JuiceFS. Our counterpart figures keep those baselines and change the denominator to ChunkTevict; object-level SkyStore appears as an extra row so the gap versus the original system is explicit. TTL is SkyStore's `calculate_ttl`; ChunkTevict only changes the unit from objects to unique chunks.

TABLE I
TWO-REGION FB COST (USD WITHOUT REQUEST), SKYSTORE FIG. 5 POLICY SET.

Trace	Evict	Store	Teven	TTL-CC	EWMA	AWS-MR	ChunkT.
T15	92.79	141.78	95.95	92.40	92.79	179.33	25.22
T29	4.60	7.15	5.02	5.57	4.60	8.48	0.39
T65	55.06	4.39	6.32	4.86	15.91	4.41	2.01
T78	62.91	24.02	17.81	19.21	22.90	28.96	2.50
T79	356.37	540.17	340.83	343.25	339.72	570.71	51.92

TABLE II
TWO-REGION FB COST / CHUNKTEVICT (SKYSTORE FIG. 5 BASELINES).

Trace	Evict	Store	Teven	TTL-CC	EWMA	AWS-MR	ChunkT.
T15	3.68×	5.62×	3.80×	3.66×	3.68×	7.11×	1.00×
T29	11.75×	18.26×	12.83×	14.23×	11.75×	21.66×	1.00×
T65	27.39×	2.18×	3.14×	2.42×	7.92×	2.19×	1.00×
T78	25.12×	9.59×	7.11×	7.67×	9.15×	11.57×	1.00×
T79	6.86×	10.40×	6.56×	6.61×	6.54×	11.0×	1.00×

Two- and three-region results in this section are **fixed-base (FB)**, matching SkyStore Fig. 5 and Table 4. Scale plots follow SkyStore Fig. 6 (type A cost divided by that policy's own three-region cost) on all five traces, including T29 at six and nine regions. Free placement uses SkyStore Table 5's set (Always Evict, Always Store, JuiceFS, SkyStore, ChunkTevict). Three-region free placement is complete for five traces; six- and nine-region T29 JuiceFS cells are omitted where the 29 GB host ran out of memory.

All experiments run on an 8-vCPU, 29 GB host. CDC is performed once offline; the replay only looks up uids. Manifest metadata is 40 B per chunk reference and is excluded from the dollar bill.

C. C. Numerical Results and Discussion

1) C1. Two-region cost (SkyStore Fig. 5): SkyStore Fig. 5 is two-region fixed-base type E: y = baseline cost / SkyStore, x = Always Evict, Always Store, T_{even}, TTL-CC, EWMA, AWS Multi-Region. Tables I–II use that same x set with y = cost / ChunkTevict. AWS Multi-Region is asynchronous replication on PUT to the other region. TTL-CC is the stochastic cache TTL; EWMA uses α=0.5. Object-level SkyStore and Optimal play SkyStore Table 3's role and sit in Table III.

As shown in Fig. 3 and Tables I–III, ChunkTevict is cheaper on every trace; the alternatives are low or comparable in specific cases and become much more expensive in others—the same pattern SkyStore §6.3 reports against TevictRanges, now with unique chunks as the denominator.

On T15, TTL-CC tracks object-level SkyStore (3.66× versus 3.65× ChunkTevict): stochastic cache TTLs do not beat TevictRanges by much, and both remain several times ChunkTevict. Always Store and AWS Multi-Region pay a second replica that one-hit writes rarely reread (5.62× and 7.11×). EWMA collapses to Always Evict (3.68×).

T29 is the extreme. Objects are tiny and GETs are miss-heavy (ChunkTevict GET chunk hit 43.8%), so Always Store and AWS Multi-Region pay 18–22× ChunkTevict while

TABLE III
TWO-REGION FB VERSUS OBJECT-LEVEL SKYSTORE AND OPTIMAL (SKYSTORE TABLE 3).

Trace	SkyStore	Optimal	ChunkT.
T15	92.02 (3.65×)	90.90 (3.60×)	25.22
T29	4.62 (11.80×)	4.59 (11.73×)	0.39
T65	4.39 (2.18×)	4.56 (2.27×)	2.01
T78	16.79 (6.71×)	16.30 (6.51×)	2.50
T79	337.02 (6.49×)	323.48 (6.23×)	51.92

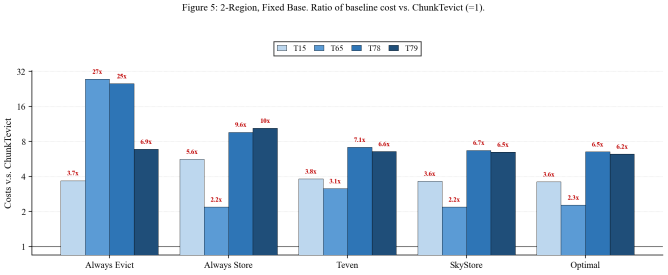


Fig. 3. Two-region FB cost / ChunkTevict for SkyStore Fig. 5 baselines (Always Evict, Always Store, T_{even}, TTL-CC, EWMA, AWS Multi-Region).

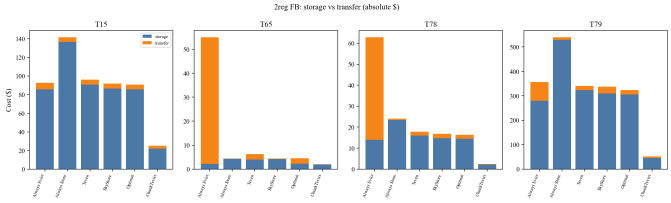


Fig. 4. Two-region FB storage versus transfer for SkyStore and ChunkTevict. Storage accounts for 88–97% of the gap.

TABLE IV
CHUNKTEVICT LOCAL CHUNK-HIT RATE (REFERENCES, NOT OBJECTS).

Setup	Trace	PUT hit	GET hit
2-reg E	T15	82.3%	55.8%
2-reg E	T29	97.0%	43.8%
2-reg E	T65	39.7%	99.8%
2-reg E	T78	87.9%	99.2%
2-reg E	T79	81.3%	88.7%
3-reg A	T15	70%	85%
3-reg A	T65	29%	99.7%
3-reg A	T78	81%	99.5%
3-reg A	T79	67%	97.5%

object-level SkyStore is still 11.80×. EWMA again matches Always Evict (11.75×).

On T65, Always Evict is catastrophic (27.39×) because hot GETs miss a remote object every time. Always Store and AWS Multi-Region sit at 2.18× and 2.19×: they buy local hits with full-object GB·h, yet still trail ChunkTevict, whose GET chunk hit is 99.8%. EWMA only partially closes the gap (7.92×). T_{even} and TTL-CC stay close to SkyStore (3.14× and 2.42× versus SkyStore 2.18×).

On T78, Always Evict is again 25.12×. T_{even} (7.11×) beats Always Store (9.59×), as SkyStore observes when a fixed network-break-even TTL is better than never evicting, but TevictRanges at object grain (6.71×) is still far from the unique-chunk bill (2.50 USD).

On T79, Always Store and AWS Multi-Region keep large objects for the whole trace (10.40× and 11.0×). TevictRanges, T_{even}, TTL-CC, and EWMA cluster around 6.5× ChunkTevict.

Object-level Optimal remains 2.27–11.73× ChunkTevict (Table III). Future knowledge of *object* accesses does not create a unique-chunk clock: Optimal still stores and moves whole keys. On T65 the artifact's Optimal ($4.56) is slightly above SkyStore ($4.39), so we treat it as a strong object-level reference rather than a strict lower bound.

2) C2. Why ChunkTevict saves (storage versus transfer): Fig. 4 splits the two-region bill. Let Δ be SkyStore minus ChunkTevict. The storage share of Δ is 96.7% (T15), 95.1% (T29), 96.5% (T65), 87.8% (T78), and 92.6% (T79). Transfer still drops—T78 SkyStore transfer is 2.08 versus ChunkTevict 0.34 USD—but most of the dollar gap is GB·h that object replicas keep alive.

Table IV reports the fraction of chunk references that find a local uid. GET chunk hits exceed 99% on T65/T78 in two regions and in every three-region type A cell we report; two-region T15 is harder (GET 55.8%) and T29 is miss-heavy (GET 43.8%) yet still beats object-level SkyStore by 11.80× because PUTs reuse 97.0% of chunks and storage is charged per uid. Object-level Optimal cannot record those hits: a miss is all-or-nothing on the key.

3) C3. Simulated GET RTT: Table V reports two-region T65 and T78 (hot versus high-volume) from the same replay as C1. Same-region hits are 1 ms; a miss is the paid-edge RTT (68.3 ms on this pair); ChunkTevict takes the max over missing chunks fetched in parallel.

On T65, ChunkTevict's average GET is 1.6 ms with p99 = 1 ms, against SkyStore 1.7 ms / p99 68.3 ms: object misses still pay the full RTT even when most chunks would have hit. Always Evict and EWMA stay at the miss RTT (68.3 ms and 44.0 ms average). Always Store and AWS Multi-Region sit at 1–1.5 ms because every object is already local—they buy that latency with the storage bill in Table I. T78 is noisier: ChunkTevict 6.2 ms average matches SkyStore 6.3 ms; both still have a 68.3 ms p99 from cold unique chunks.

4) C4. Three-region workload types (SkyStore Table 4): SkyStore Table 4 reports, for each policy, the mean over traces of (policy cost)/(SkyStore cost), with rows Always Evict, Always Store, T_{even}, TTL-CC, EWMA, and JuiceFS. We keep that row set and the type A–D columns, change the denominator to ChunkTevict, and add a SkyStore row (Table VI, Fig. 5).

Averaged over the five traces, SkyStore is 6.40× ChunkTevict (type B 6.29×, type A 6.43×). T29 dominates the mean: ChunkTevict is $0.36 on three-region T29 while SkyStore remains $5.3–5.7, so that one trace is about 15× by itself. JuiceFS is 14.91× on average—SkyStore Table 4 already reported JuiceFS 1.9–4.8× SkyStore; we see the same industrial baseline after changing the denominator. TTL-CC

TABLE V
TWO-REGION SIMULATED GET RTT (MS) FROM THE TRANSFER-GRAPH EDGE. PUT IS 1 MS FOR EVERY POLICY.

	GET T65			GET T78		
Policy	avg	p90	p99	avg	p90	p99
Always Evict	68.3	68.3	68.3	68.3	68.3	68.3
Always Store	1.5	1.0	1.0	2.7	1.0	68.3
T_{even}	9.5	68.3	68.3	5.8	1.0	68.3
TTL-CC	3.7	1.0	68.3	5.5	1.0	68.3
EWMA	44.0	68.3	68.3	24.6	68.3	68.3
AWS-MR	1.0	1.0	1.0	1.0	1.0	1.0
SkyStore	1.7	1.0	68.3	6.3	1.0	68.3
ChunkTevict	1.6	1.0	1.0	6.2	1.0	68.3

TABLE VI
THREE-REGION FB: MEAN OVER FIVE TRACES OF COST / CHUNKTEVICT, SKYSTORE TABLE 4 WORKLOAD TYPES.

Policy	A unif.	B region	C agg.	D repl.	Mean
Always Evict	23.67×	42.00×	34.20×	25.60×	31.37×
Always Store	8.38×	7.14×	7.41×	8.43×	7.84×
T_{even}	7.19×	6.66×	6.82×	7.23×	6.98×
TTL-CC	7.60×	6.49×	6.78×	7.71×	7.15×
EWMA	10.15×	10.81×	10.16×	10.35×	10.37×
JuiceFS	14.29×	15.42×	15.63×	14.31×	14.91×
SkyStore	6.43×	6.29×	6.43×	6.45×	6.40×
ChunkTevict	1.00×	1.00×	1.00×	1.00×	1.00×

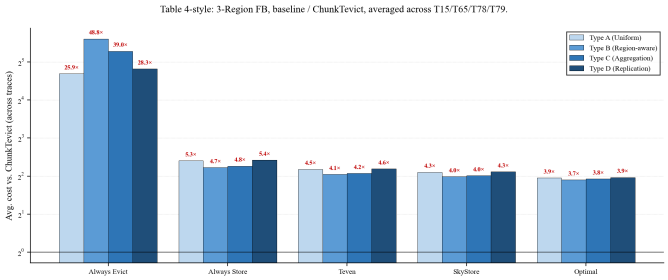


Fig. 5. Three-region FB mean cost / ChunkTevict by SkyStore Table 4 workload type (five traces).

TABLE VII
THREE-REGION FP: MEAN OVER FIVE TRACES OF COST / CHUNKTEVICT (SKYSTORE TABLE 5 POLICY SET).

Policy	A unif.	B region	C agg.	D repl.	Mean
Always Evict	21.39×	50.18×	44.45×	23.21×	34.81×
Always Store	5.70×	6.03×	8.71×	5.75×	6.55×
JuiceFS	9.52×	12.94×	18.34×	9.53×	12.58×
SkyStore	4.52×	4.41×	6.55×	4.53×	5.00×
ChunkTevict	1.00×	1.00×	1.00×	1.00×	1.00×

tracks SkyStore (7.15× versus 6.40×). EWMA sits between Always Evict and T_{even} (10.37×). Type B (region-aware) inflates Always Evict the most (42.00×) because misses concentrate on expensive edges, while ChunkTevict hits unique chunks locally. Type A (uniform) is the mildest Evict penalty (23.67×).

5) C5. Scaling the number of regions (SkyStore Fig. 6): Table VIII reports type A cost divided by *that policy's own* three-region cost, matching SkyStore Fig. 6. Over five traces, SkyStore's mean 6-region / 3-region ratio is 1.25 and 9-region / 3-region is 1.52; ChunkTevict is 1.53 and 2.06. T15, T29, and T79 stay relatively flat for SkyStore (1.03–1.28), as in the original Fig. 6; object-level T29 barely grows ($5.35 → $5.50 → $5.67) because write-heavy keys already pay an immortal origin. ChunkTevict on T29 grows 1.72× / 2.39× because unique chunks become resident in more caches—the same mechanism that saves dollars in two and three regions. T65 still grows fastest for both systems (ChunkTevict 2.84× at nine regions) as uniform GETs spread. JuiceFS type A is 2.05× and 3.17× its own three-region bill at six and nine regions (T15 312.87 → 640.97 → 990.36), matching SkyStore Table 5's warning that replicate-on-write storage scales with region count. Nine-region JuiceFS T29 is missing (out-of-memory on the evaluation host).

6) C6. Free placement (SkyStore Table 5): SkyStore Table 5 evaluates free placement (FP), where no region holds an immortal origin copy. We use its policy set—Always Evict, Always Store, JuiceFS—and add SkyStore and ChunkTevict (Table VII). Averaged over five traces and four types, SkyStore is 5.00× ChunkTevict, lower than the FB mean of 6.40×. The drop is consistent with SkyStore §6.5: without a pinned base, object-level TevictRanges already evicts more aggressively. The remaining gap is chunk billing.

Type C is the outlier (6.55×), driven by T29: ChunkTevict costs $0.34 there against SkyStore's $4.87. On the other T29 types, ChunkTevict under FP is *more* expensive than under FB (type A: $1.24 versus $0.36; type B: $0.91). Without a pinned origin, chunks shared by many small objects expire and must be fetched back from a remaining holder, so for write-heavy traces with heavy reuse the pinned origin is worth more to ChunkTevict than to SkyStore. Six- and nine-region FP JuiceFS T29 cells exceeded the host's memory and are omitted.

7) C7. Overall time: Table IX lists wall-clock replay time and operations per second on the 8-vCPU host—the counterpart of overall time in [11], not client GET/PUT latency. The heavy traces are T78 (2.41 million operations, 3.1 h in two regions) and T29 (13.0 million operations, 12.9 h in two regions). T79 is only 114 thousand operations but 41 ops/s in two regions because objects are large (many uids per request). Three-region T29 is much faster per operation than two-region (3670 versus 280 ops/s) because unique chunks stay hot across three caches. CDC is offline and is not on the replay critical path. The unique-chunk indexes used by the replay contain 8.63 M, 0.13 M, 0.25 M, 0.67 M, and 8.69 M uids for T15/T29/T65/T78/T79; each chunk reference occupies 40 B in the manifest and is not added to the storage bill.

D. D. Threats to Validity

Content binding. The IBM traces carry no payload, so the cross-object sharing that ChunkTevict exploits comes from our binding to versioned public data. Workloads with little shared content (encrypted, compressed, or unique media objects) would reduce ChunkTevict to object-level SkyStore plus chunk

TABLE VIII
TYPE A FB COST / OWN 3-REGION COST (SKYSTORE FIG. 6). FIVE TRACES.

Policy	Trace	3-reg $	6/3	9/3
SkyStore	T15	100.05	1.03×	1.06×
	T29	5.35	1.03×	1.06×
	T65	6.85	1.87×	2.73×
	T78	27.84	1.21×	1.45×
	T79	398.18	1.10×	1.28×
	mean	—	1.25×	1.52×
ChunkTevict	T15	36.69	1.26×	1.45×
	T29	0.36	1.72×	2.39×
	T65	3.06	1.89×	2.84×
	T78	4.22	1.38×	1.77×
	T79	71.18	1.39×	1.84×
	mean	—	1.53×	2.06×

TABLE IX
CHUNKTEVICT REPLAY WALL TIME ON THE 8-VCPU HOST.

Setup	Trace	Ops	Wall (s)	Ops/s	Cost
2-reg E	T15	1 575 627	5764	273	25.22
2-reg E	T29	13 019 220	46457	280	0.39
2-reg E	T65	283 086	2517	112	2.01
2-reg E	T78	2 409 805	11250	214	2.50
2-reg E	T79	114 227	2814	41	51.92
3-reg A	T15	1 575 627	2575	612	36.69
3-reg A	T29	13 019 220	3547	3670	0.36
3-reg A	T65	283 086	1486	191	3.06
3-reg A	T78	2 409 805	8528	283	4.22
3-reg A	T79	114 227	760	150	71.18

metadata; the absolute ratios should be read as conditional on content that repeats.

Two changes at once. ChunkTevict both stores each chunk once per region and gives each chunk its own lease. Our baselines bill whole objects, so the reported ratios include both effects; Section V-C2 shows that most of the gap is storage.

Request charges and metadata. As in SkyStore, cost excludes per-request charges. A chunk-level store issues more GET/PUT calls unless chunks are packed into containers, and the chunk index itself occupies $40\,\mathrm{B}$ per reference. Latency is a lookup on the transfer graph, not a live multi-cloud measurement.

VI. CONCLUSION

With the prevalence of public-cloud object storage, geo-distributed clients pay for bytes stored over time and bytes moved across regions. This paper reviews object-granularity TevictRanges and analyzes its limitation: when objects share content, SkyStore and even its object-level Optimal store and move the same chunks under independent object leases. Thus, we propose ChunkTevict, which keeps the TevictRanges histogram, T_{evict} estimator, and TTL rule, and moves placement, eviction, transfer, and GB·h accounting to unique chunks per region. Finally, we implement an offline chunk index and a trace replay, and evaluate the five IBM traces of SkyStore Fig. 5 bound to versioned public content. The results show that two-region SkyStore costs 2.18–$11.80\times$ ChunkTevict and object-level Optimal 2.27–$11.73\times$, with 88–97% of the gap in storage. In three regions, SkyStore averages $6.40\times$ ChunkTevict with a fixed base and $5.00\times$ under free placement, so the gap is not an artifact of pinning the origin copy. How much of this carries over to production depends on how much content real workloads share and on request charges for chunk-level access; measuring both on payload-bearing traces is our next step.

REFERENCES

[1] S. Liu, X. Mo, M. Hershcovitch, H. Zhang, A. Cheng, G. Girmonsky, G. Vernik, M. Factor, T. Bang, S. Ponnapalli, N. Crooks, J. E. Gonzalez, D. Harnik, and I. Stoica, "SkyStore: Cost-optimized object storage across regions and clouds," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5004–5017, 2025, arXiv:2502.20818.

[2] O. Eytan, D. Harnik, E. Ofer, R. Friedman, and R. Kat, "It's time to revisit IBM object storage traces," in *Proceedings of the 12th USENIX Workshop on Hot Topics in Storage and File Systems*, 2020, SNIA IOTTA trace set 36305.

[3] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles*, 2001, pp. 174–187.

[4] M. O. Rabin, "Fingerprinting by random polynomials," *Harvard University Center for Research in Computing Technology*, 1981, technical Report TR-15-81.

[5] Z. Wu, M. Butkiewicz, D. Perkins, E. Katz-Bassett, and H. V. Madhyastha, "SPANStore: Cost-effective geo-replicated storage spanning multiple cloud services," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, 2013, pp. 292–308.

[6] S. Agarwal, J. Dunagan, N. Jain, S. Saroiu, A. Wolman, and H. Bhogan, "Volley: Automated data placement for geo-distributed cloud services," in *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation*, 2010.

[7] M. Uluyol, A. Huang, A. Goel, M. Chowdhury, and H. V. Madhyastha, "Near-optimal latency versus cost tradeoffs in geo-distributed storage," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 2020, pp. 157–180.

[8] D. Carra, G. Neglia, and P. Michiardi, "TTL-based cloud caches," in *IEEE INFOCOM*, 2019, pp. 685–693.

[9] Y. Zhang, W. Xia, D. Feng, H. Jiang, Y. Hua, and Q. Wang, "Finesse: Fine-grained feature locality based fast resemblance detection for post-deduplication delta compression," in *Proceedings of the 17th USENIX Conference on File and Storage Technologies*, 2019, pp. 121–137.

[10] W. Xia, H. Jiang, D. Feng, and L. Tian, "DARE: A deduplication-aware resemblance detection and elimination scheme for data reduction with low overheads," in *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014, extended in IEEE Trans. Computers 2016.

[11] X. Ye, J. Tang, W. Tian, R. Li, W. Xiao, Y. Geng, and Z. Xu, "Fast variable-grained resemblance data deduplication for cloud storage," in *IEEE International Conference on Communications*, 2021.